Databricks Best Practices

A practical guide to building performant, cost-efficient, and reliable data pipelines on Databricks — for data engineers and analytics engineers working at scale.

ENGINEERING EXCELLENCE

DATABRICKS & SPARK

Why Best Practices Matter

In Databricks environments, architectural and configuration decisions have a direct, measurable impact on three critical dimensions. Ignoring them leads to slow pipelines, runaway costs, and brittle workflows that fail under load.

Performance

Poorly optimised jobs consume more resources and take longer to run. Smart partitioning, caching, and file formats can reduce query times dramatically.

Cost

Compute and storage are not free. Oversized clusters, inefficient file formats, and full reloads drive up DBU consumption unnecessarily.

Reliability

Idempotent, incremental pipelines that handle failures gracefully ensure your data platform is dependable and production-ready at all times.

File Format Best Practices

The file format you choose underpins everything — from query speed to storage efficiency to ACID compliance. Getting this right is one of the highest-leverage decisions in your data architecture.

✅ Use Delta & Parquet

Delta Lake is the default choice on Databricks. It provides ACID transactions, schema enforcement, time travel, and efficient data skipping — all critical for production workloads.

Parquet is the underlying columnar format offering excellent compression and predicate pushdown for analytical queries.

❌ Avoid CSV in Production

CSV files have no schema, no compression, no support for partial reads, and are slow to query at scale. They lack ACID guarantees and cannot support incremental processing efficiently.

CSV is acceptable only for raw ingestion at the boundary of your system — convert to Delta as early as possible in your pipeline.

Partitioning Strategies

Partitioning allows Spark to skip large volumes of data at read time by organising files into logical directories. Done well, it is one of the most impactful performance optimisations available.

Partition Large Datasets Thoughtfully

Choose partition columns with **low-to-medium cardinality** — common choices include `date`, `region`, or `status`. This ensures Spark can skip entire partitions during query execution, reducing shuffle and I/O significantly.

Avoid Over-Partitioning

Over-partitioning (e.g., partitioning by a high-cardinality column like `user_id`) creates millions of tiny files — the infamous **"small files problem"**. This degrades performance due to excessive metadata overhead and Spark task scheduling costs. A good rule of thumb: aim for partition files of at least 128 MB.

💡 Use `OPTIMIZE` and `VACUUM` regularly on Delta tables to compact small files and remove stale data versions.

Caching in Spark

When to Cache

Caching is most valuable when a DataFrame is **reused multiple times** within the same job or notebook session. Use `.cache()` or `.persist()` to store intermediate results in memory, avoiding repeated recomputation of expensive transformations such as joins and aggregations.

- Iterative ML training loops
- Multiple downstream consumers of the same transform
- Interactive exploration of large, pre-filtered datasets

When Not to Cache

Indiscriminate caching is one of the most common causes of out-of-memory errors and degraded cluster performance. Cached data occupies executor memory and evicts other data.

- Do not cache DataFrames used only once
- Always call `.unpersist()` when data is no longer needed
- Avoid caching very large datasets that exceed available memory

Z-Ordering: Data Skipping at Scale

Z-Ordering is a Delta Lake technique that co-locates related data within the same set of files, enabling Databricks to skip irrelevant files based on column statistics during query execution. It is complementary to partitioning and especially effective for high-cardinality filter columns.

Optimize Collect Stats Skip Files

Use Z-Ordering on columns frequently used in `WHERE` clauses and joins — such as `customer_id`, `event_type`, or `product_sku`. Combine with `OPTIMIZE` to compact files first. Z-Ordering is most impactful on tables with billions of rows where full scans are prohibitively expensive.

Cluster Optimisation

Your cluster configuration is a primary cost and performance lever. Right-sizing your cluster — and choosing the correct instance strategy — can reduce DBU costs by 40–60% without any code changes.

Autoscaling

Enable autoscaling to allow Databricks to dynamically add or remove workers based on workload demand. This prevents both under-provisioning (slow jobs) and over-provisioning (wasted spend). Set sensible min/max worker bounds to control costs.

Instance Types

Match instance families to workload profiles. Memory-optimised instances suit large shuffle operations and caching. Compute-optimised instances suit CPU-intensive transformations. Avoid general-purpose instances for specialised workloads.

Spot Instances

Use spot (preemptible) instances for worker nodes to reduce compute costs by up to 70%. Reserve on-demand instances for the driver node only, ensuring job orchestration is never interrupted by spot reclamation.

Job Clusters vs. Interactive Clusters

Choosing the right cluster type for each use case is essential for both cost control and performance. Using interactive clusters for production jobs is one of the most common and costly mistakes in Databricks deployments.

Attribute	Job Cluster	Interactive (All-Purpose) Cluster
Lifecycle	Spun up and torn down automatically per job run	Long-running, persistent across sessions
Best For	Scheduled pipelines, automated ETL, CI/CD workflows	Notebook exploration, ad hoc analysis, development
Cost Profile	Lower — billed only for job duration	Higher — billed for entire uptime including idle time
Isolation	Fully isolated per run — no cross-job interference	Shared state can cause conflicts between users
Recommendation	✅ Use for all production workloads	⚠️ Use only for development and exploration

💡 Always terminate interactive clusters when not in use. Set auto-termination to 30–60 minutes to prevent idle cost accumulation.

Pipeline Best Practices

Robust data pipelines are designed to be safe to re-run, efficient in what they process, and resilient to failure. These three principles should guide every pipeline you build on Databricks.

Idempotency

Design pipelines so that running them multiple times produces the same result. Use `MERGE INTO` (upserts) in Delta Lake to safely handle duplicate runs without corrupting data.

Incremental Processing

Use Delta Lake's Change Data Feed or Structured Streaming to process only new or changed records. Incremental loads reduce compute consumption and improve SLA compliance significantly.

Avoid Full Reloads

Full table scans and rewrites are expensive and fragile. Reserve full reloads for initial backfills or schema migrations only. For everything else, process incrementally from a reliable watermark or checkpoint.

The Guiding Principle

> "Performance and cost are design decisions, not afterthoughts."

Every choice you make — from file format to cluster type to partitioning strategy — has downstream consequences on speed, reliability, and spend. The best Databricks practitioners embed these considerations from the first line of pipeline design, not as a bolt-on optimisation at the end.

📁 Use Delta

Always prefer Delta over CSV or raw Parquet in production

⚙️ Right-size Clusters

Autoscale, spot workers, job clusters for production

🔄 Build Incrementally

Idempotent, watermarked, failure-safe pipelines

🔍 Optimise Reads

Partition wisely, Z-Order high-cardinality columns